

Auditoria de Repositório, Deploy e Documentação

Projeto: GC Sistema (Gestão de Contratos) · **Repositório:** github.com/cremalet/gc-sistema · **Data:** 01/09/2026 · **Responsável pela verificação:** Breno (breno@obraminds.com)

1. Resumo executivo

Dos seis itens do checklist, **um foi corrigido** nesta rodada (README), **três foram confirmados como já em conformidade** (sincronização com o remoto, `.env.local` fora do versionamento, deploy na Vercel com build verde) e **dois estão bloqueados por falta de permissão de acesso** (visibilidade do repositório e configuração/teste em produção na Vercel).

O ponto crítico continua aberto: **o repositório está PÚBLICO**. A conta que executou a verificação (`brenocml0`) tem apenas permissão de leitura (`pull`) no repositório, que pertence à conta pessoal `cremalet` — a alteração de visibilidade exige permissão de `admin` e só pode ser feita pelo dono.

Limite de escopo desta auditoria. Os dois itens em aberto não são trabalho pendente da equipe de desenvolvimento: são bloqueios de acesso. Quem executou a verificação não tem permissão de `admin` no GitHub nem acesso ao projeto na Vercel (`cremalets-projects`) ou credenciais de usuário de `gc-prod`. Ambos precisam ser executados por quem detém esses acessos — não há contorno técnico do lado do repositório local.

Mitigação já verificada: a varredura do histórico completo dos 17 commits não encontrou nenhum segredo vazado — nenhum arquivo `.env` com valores foi versionado em qualquer momento, e não há chaves JWT/ `service_role` em nenhuma revisão. O que está exposto pela visibilidade pública é a **modelagem financeira e as regras de negócio** (percentuais de sinal/FD/medição, regras de conciliação de Faturamento Direto, políticas de acesso por perfil), não credenciais.

2. Situação por item do checklist

Atividade	Situação	Constatação
Tornar o repositório privado	PENDENTE	Repositório PUBLIC (<code>isPrivate: false</code>). A tentativa de alteração via <code>gh repo edit --visibility private</code> retornou <code>HTTP 404</code> : a conta <code>brenocml0</code> tem permissões <code>{admin: false, push: false, pull: true}</code> . Depende do dono <code>cremalet</code> .
Cópia local sincronizada com o remoto	OK RESSALVA	<code>git rev-list --left-right --count origin/main...HEAD = 0 0</code> — mesmo commit (<code>1295af8</code>), 17 commits, batendo com o remoto. Ressalva: há arquivos não commitados (ver §3).
<code>.env.local</code> não versionado e <code>.env.local.example</code> atualizado	OK	<code>git ls-files grep env</code> retorna apenas <code>.env.local.example</code> . <code>.gitignore</code> cobre <code>.env*.local</code> . O <code>example</code> documenta as duas variáveis efetivamente usadas pelo código mais a <code>SUPABASE_SERVICE_ROLE_KEY</code> (reservada, ainda não consumida).
Projeto deployado na Vercel com build verde	OK	11 deployments de Production criados por <code>vercel[bot]</code> , um por commit em <code>main</code> . O último (<code>1295af8</code>) está com <code>state: success</code> . Build de produção reproduzido localmente com sucesso (28 rotas + middleware).
Env vars de gc-prod na Vercel e login testado em produção	SEM PERMISSÃO	Fora do alcance de quem verificou — sem acesso ao projeto na Vercel (<code>cremalets-projects</code>) nem credenciais de usuário de gc-prod. O que foi possível confirmar por fora, sem autenticar: as env vars já estão configuradas e apontam para um projeto Supabase distinto do de desenvolvimento (<code>zondscnfosubxutpshux</code> em produção vs. <code>gzbmhgnpoehormnidmgg</code> usado pelo <code>db:types</code>); a tela de login responde <code>200</code> e a raiz redireciona <code>307 → /login</code> . Falta apenas o teste de login autenticado.
Substituir o README template do create-next-app	CORRIGIDO	README template (36 linhas, boilerplate do <code>create-next-app</code>) substituído por documentação real do projeto (172 linhas). Conteúdo em §4.

3. Evidências da verificação

3.1 Visibilidade e permissões

Visibilidade	PUBLIC
Dono	cremalet (conta de usuário, não organização)
Permissões de brenocml0	admin: false · maintain: false · push: false · triage: false · pull: true
Criado / último push	24/04/2026 · 12/05/2026
Commits	17

Como brenocml0 não tem push, a branch local feature-dev-breno-automacao também não pode ser publicada no remoto sem ajuste de permissão ou uso de fork.

3.2 Varredura de segredos no histórico

Executada em todas as revisões (git rev-list --all), buscando tokens JWT (eyJ...), sb_secret e atribuições de service_role, além do diff completo de qualquer arquivo .env*: **nenhuma ocorrência**. O único .env* já versionado é o .env.local.example, com todos os valores em branco desde o primeiro commit.

3.3 Estado da árvore de trabalho

A branch está sincronizada, mas há trabalho não commitado — inclusive três migrations de banco:

Arquivo não versionado	Observação
supabase/008_documentos_processamento.sql	migration de schema fora do controle de versão
supabase/009_contatos_whatsapp.sql	migration de schema fora do controle de versão
supabase/010_storage_bucket_documentos_processamento.sql	migration de schema fora do controle de versão
pnpm-lock.yaml e pnpm-workspace.yaml	segundo gerenciador de pacotes; o lockfile versionado é o package-lock.json
.token-optimizer/	cache de ferramenta local; deve entrar no .gitignore

Risco: as migrations 008–010 existem apenas na máquina local. Se o banco de gc-prod já as tiver aplicado, o repositório deixou de ser a fonte de verdade do schema; se não, o código em main e o banco divergem. Dois lockfiles no mesmo projeto também abrem espaço para a Vercel resolver dependências diferentes das da máquina local.

3.4 Deploy e produção

Integração	Vercel conectada ao GitHub; Production a cada push em <code>main</code>
Último deployment	<code>1295af8</code> — 12/05/2026 18:21 — <code>state: success</code>
URL de produção	<code>https://gc-sistema.vercel.app</code> (alias público, HTTP 200)
URLs por deployment	protegidas por Vercel SSO (redirecionam para <code>vercel.com/sso-api</code>)
Supabase de produção	<code>zondscnfosubxutpshux.supabase.co</code> — distinto do projeto de desenvolvimento
Build local	<code>next build</code> concluído sem erros — 28 rotas + middleware (84.9 kB)
CI	ausente — não há <code>.github/workflows</code> ; lint e build não são verificados antes do merge

O fato de a página `/login` renderizar e de a raiz redirecionar via middleware confirma que `NEXT_PUBLIC_SUPABASE_URL` e `NEXT_PUBLIC_SUPABASE_ANON_KEY` estão presentes no ambiente de produção — sem elas o middleware falharia na inicialização. Toda a constatação acima foi obtida por inspeção externa (API pública do GitHub e requisições HTTP ao domínio de produção); **não houve acesso ao painel da Vercel**, portanto não foi possível auditar a lista completa de env vars, seus escopos por ambiente nem executar o login autenticado.

4. Correção aplicada: README

O `README.md` continha apenas o boilerplate do `create-next-app`. Foi reescrito como documentação de onboarding do projeto, cobrindo:

- **Aviso de confidencialidade** — repositório privado, sem dumps ou chaves em issues/PRs.
- **Stack** — Next.js 14 (App Router, TS strict), Tailwind 3, Supabase (`@supabase/ssr`), react-hook-form + zod, exceljs, `@react-pdf/renderer`, Vercel.
- **Setup local** — `npm install`, cópia do `.env.local.example`, scripts `dev / build / lint / db:types`.
- **Variáveis de ambiente** — tabela de cada variável, onde é usada, se é obrigatória, e a regra de atualizar o *example* no mesmo commit.
- **Ambientes** — separação `gc-dev / gc-prod` e onde ficam as env vars de produção.
- **Arquitetura** — mapa de `src/app`, `src/components`, `src/lib` e `supabase/`.
- **Autenticação e autorização** — middleware, `requirePerfil()`, filtro de menu por perfil e RLS como última linha de defesa; os seis perfis e a matriz módulo × perfil.
- **Domínio** — fluxo Orçamento → Proposta → Contrato → Obra, itens/execução, financeiro e a mecânica do FD (`valor`, `valor_descontar`, `diferenca_favor` como coluna gerada); views principais.
- **Banco de dados** — tabela das migrations 000–010 com a finalidade de cada uma, seeds e o passo `db:types` após mudança de schema.
- **Deploy e convenções** — publicação pela Vercel, `lint + build` antes do PR, e as convenções do código (domínio em português, alias `@/*`, Server Components por padrão, toda tabela nova com

`empresa_id` + RLS no mesmo commit).

5. Ações requeridas

#	Ação	Responsável	Prioridade
1	Tornar o repositório privado: <i>Settings</i> → <i>Danger Zone</i> → <i>Change visibility</i>	dono <code>cremalet</code>	CRÍTICA bloqueia acelerar o desenvolvimento
2	Conceder <code>push</code> (ou <code>maintain</code>) a <code>brenocml</code> no GitHub, hoje somente leitura, e acesso ao projeto na Vercel	dono <code>cremalet</code>	ALTA bloqueia contribuição e a verificação de produção
3	Commitar as migrations 008–010 para que o repositório volte a ser a fonte de verdade do schema	Breno (após #2)	ALTA
4	Revisar no painel da Vercel a lista completa de env vars de gc-prod (escopo por ambiente) e testar o login em produção com um usuário real, validando o redirecionamento por perfil	quem tem acesso à Vercel (<code>cremalets-projects</code>) e credenciais de gc-prod — não é executável por <code>brenocml</code>	MÉDIA
5	Decidir o gerenciador de pacotes: remover <code>pnpm-lock.yaml</code> / <code>pnpm-workspace.yaml</code> ou migrar e remover o <code>package-lock.json</code>	Breno	MÉDIA
6	Adicionar CI mínimo no GitHub Actions (<code>npm ci</code> + <code>lint</code> + <code>build</code>) em pushes e PRs	Breno	MÉDIA
7	Adicionar <code>.token-optimizer/</code> ao <code>.gitignore</code>	Breno	BAIXA
8	Consumir ou remover <code>SUPABASE_SERVICE_ROLE_KEY</code> do <code>.env.local.example</code> — hoje está documentada mas não é usada pelo código	Breno	BAIXA

Recomendação sobre o gate de aceleração. O deploy está confirmado e verde, e a documentação foi entregue — dois dos três pré-requisitos estão atendidos. O que resta são exclusivamente **ações de titular de acesso**: tornar o repositório privado e revisar/testar o ambiente de produção na Vercel. Nenhuma das duas depende do time de desenvolvimento, e nenhuma tem contorno técnico do lado local. Como não há segredos no histórico, tornar o repositório privado resolve a exposição integralmente, sem necessidade de reescrita de histórico ou rotação de chaves. Recomenda-se conceder os acessos da ação #2 junto com a #1, para que as verificações futuras não voltem a parar no mesmo ponto.

Relatório gerado a partir de verificação direta no repositório local, na API do GitHub e no ambiente de produção na Vercel.
Comandos e respostas registrados na sessão de origem.